SO SÁNH BỐN THUẬT TOÁN ĐIỀU KHIỂN ĐÈN GIAO THÔNG

Fixed-Time — MaxPressure — Q-Learning — Deep Q-Learning

Trên nền tảng mô phỏng GAMA kết hợp PyTorch

Báo cáo này tổng hợp quá trình triển khai và kết quả đánh giá so sánh bốn kỹ thuật điều khiển đèn tín hiệu tại một ngã tư bốn hướng mô phỏng trên nền tảng GAMA: (i) điều khiển thời gian cố định (**Fixed-Time**), (ii) điều phối theo áp lực tối đa (**MaxPressure**), (iii) học tăng cường dạng bảng (**Q-Learning**) và (iv) học tăng cường sâu (**Deep Q-Learning**). Việc đánh giá được thực hiện trên sáu tiêu chí hiệu năng giao thông; mỗi thuật toán chạy năm lần độc lập với cùng luồng sinh xe để bảo đảm tính công bằng và khả năng lặp lại.

1 Cách triển khai

1.1 Nền tảng mô phỏng chung

Cả bốn thuật toán được cài đặt trong bốn tệp GAML riêng biệt (`Traffic/models/intersection-*.gaml`) nhưng cùng nạp một tệp mô hình cơ sở dùng chung `Traffic/includes/collision_avoidance.gaml`. Nhờ đó toàn bộ hạ tầng mô phỏng — mạng đường, logic di chuyển của xe, điểm sinh xe, cách tính chỉ số — đồng nhất tuyệt đối giữa bốn mô hình, chỉ khác nhau ở bộ điều khiển đèn:

- **Ngã tư bốn hướng**, mỗi chiều có 6 làn xe; bước mô phỏng 0,1 giây; 14 đoạn đường gồm 4 đường tiếp cận, 8 cung rẽ trong ngã tư và 2 đường đi thẳng qua.
- **Xe mô phỏng** gồm hai loại: ô tô (40%, dài 4,5 m, chiếm 2 làn, 25–40 km/h) và xe máy (60%, dài 2,5 m, chiếm 1 làn, 20–35 km/h). Xe rời chọn tuyến theo xác suất 50% đi thẳng / 25% rẽ trái / 25% rẽ phải, di chuyển theo mô hình bám xe IDM (Intelligent Driver Model).
- **Lưu lượng bất đối xứng**: trục Bắc–Nam sinh 1 xe mỗi 15 chu kỳ (1,5 giây), trục Đông–Tây 1 xe mỗi 45 chu kỳ (4,5 giây) — tạo tình huống kiểm thử khó cho bộ điều khiển.
- **Tín hiệu chia 4 pha toàn cục**: `global_phase` $\in$ {NS-GREEN, NS-YELLOW, EW-GREEN, EW-YELLOW}; bốn nút tiếp cận cùng đọc biến toàn cục này để hiển thị đèn.
- **Sáu chỉ số hiệu năng** được cập nhật mỗi chu kỳ và xuất ra stdout dưới dạng dòng `METRIC_CSV` mỗi 10 chu kỳ: throughput, thời gian chờ trung bình, chiều dài hàng đợi (xe có tốc độ < 1 km/h), tốc độ trung bình, thời gian hành trình trung bình và thời gian chờ tối đa. Mỗi lần chạy batch kéo dài 2000 chu kỳ (200 giây mô phỏng).

1.2 Fixed-Time — điều khiển thời gian cố định

Tệp cài đặt: `intersection-fixed-time.gaml`. Đây là bộ điều khiển baseline, hoàn toàn không quan sát trạng thái giao thông. Chu kỳ đèn gồm bốn pha cố định $[30, 3, 30, 3]$ giây (xanh B–N $\rightarrow$ vàng $\rightarrow$ xanh Đ–T $\rightarrow$ vàng), lặp lại mỗi 66 giây. Bộ điều khiển chỉ là một máy trạng thái đếm thời gian trong reflex `traffic_light_controller`:

```
// reflex traffic_light_controller - moi buoc mo phong 0.1s
global_counter += step;
if (global_counter >= phase_durations[global_phase]) {
    global_counter = 0;                          // reset bo dem cua pha
    global_phase = (global_phase + 1) mod 4;  // chuyen pha ke tiep
}
// phase_durations = [30s, 3s, 30s, 3s] - khai bao tinh, khong doi
```

Cách triển khai này đơn giản, tất định và dễ cấu hình, nhưng vì thời lượng pha không phụ thuộc lưu lượng nên trong điều kiện demand bất đối xứng (trục đông, trục vắng) sẽ xuất hiện lãng phí thời gian xanh ở trục vắng và dồn ứ ở trục đông.

1.3 MaxPressure — điều phối theo áp lực tối đa

Tệp cài đặt: `intersection-maxpressure.gaml`, theo thuật toán Max-Pressure của Varaiya [1]. Nguyên lý: mỗi trục (B–N và Đ–T) có một "áp lực" bằng *số xe đang trên các đường tiếp cận trừ số xe đang trên các đường rời*; pha xanh luôn ưu tiên trục có áp lực lớn hơn. Các danh sách đường vào/ra cho từng trục được nạp một lần vào biến toàn cục (`ns_incoming_roads`, `ew_outgoing_roads`,…) để việc đếm xe trong reflex `maxpressure_controller` chạy nhanh.

Bộ điều khiển chỉ ra quyết định trong pha xanh và bị ràng buộc bởi bốn ngưỡng an toàn:

- `min_green_time` = 10 s: thời gian xanh tối thiểu, tránh đổi pha quá sớm;
- `check_interval` = 5 s: chu kỳ đánh giá áp lực;
- `max_green_time` = 60 s: buộc đổi pha (`force_switch`) để tránh bỏ đói trục kia (starvation);
- `yellow_time` = 3 s: pha vàng chuyển tiếp cố định, không AI can thiệp.

Điểm cốt lõi của thuật toán là quy tắc trễ (hysteresis) với biên $\delta = 2{,}0$ xe: pha xanh chỉ bị rút ngắn khi áp lực trục đối diện vượt trội *đủ lớn*, nhằm chống hiện tượng dao động pha (anti-flicker) khi hai trục cân bằng:

$$P_{\mathrm{NS}} = N^{\mathrm{in}}_{\mathrm{NS}} - N^{\mathrm{out}}_{\mathrm{NS}}, \qquad P_{\mathrm{EW}} = N^{\mathrm{in}}_{\mathrm{EW}} - N^{\mathrm{out}}_{\mathrm{EW}}, \qquad \text{đổi pha} \iff P_{\text{đối diện}} > P_{\text{hiện tại}} + \delta. \quad (1)$$

```
// Reflex maxpressure_controller - chi danh gia trong PHA XANH
check_now    = (phase_timer >= min_green) and (mod 5s tick);
force_switch = (phase_timer >= max_green);
if (check_now or force_switch) {
    // Dem xe theo duong hien tai cua tung xe
    pressure_NS = in_NS - out_NS;
    pressure_EW = in_EW - out_EW;
    if (global_phase == NS_GREEN and (force_switch
            or pressure_EW > pressure_NS + 2.0)) {
        global_phase = NS_YELLOW;   // chuyen tiep qua pha vàng 3s
    }
    // ... doi xung cho pha EW_GREEN
}
```

Lỗi đã gặp và sửa: khi chạy headless, phần `init` của mô hình thực thi *trước* khi mạng đường được tạo, khiến `intersection(1)` = nil và các danh sách đường rỗng ⇒ áp suất luôn bằng 0, bộ điều khiển "mù" và chỉ còn hành vi force-switch mỗi 60 giây. Khắc phục: chuyển việc nạp danh sách đường sang reflex khởi tạo muộn `init_road_lists` (chạy khi `length(intersection) > 3`), cách làm giống mô hình DQN.

1.4 Q-Learning — học tăng cường dạng bảng

Tệp cài đặt: `intersection-qlearning.gaml`. Bộ điều khiển là một agent học tăng cường [2] với bảng Q lưu trong biến `map<string, list<float>> q_table`. Không gian được rời rạc hóa như sau:

- **Trạng thái** (18 trạng thái $= 2 \times 3 \times 3$): pha xanh hiện tại $\in \{\text{NS}, \text{EW}\}\times$ mức hàng đợi trục B–N $\in \{0,1,2\}\times$ mức hàng đợi trục Đ–T $\in \{0,1,2\}$. Hàng đợi được phân cấp theo số xe đang dừng ($v < 1$ km/h) trên đường vào: < 5 xe là mức 0, 5–14 xe là mức 1, ≥ 15 xe là mức 2. Mã hóa thành chuỗi "`<pha>_<muc NS>_<muc EW>`" làm khóa của bảng Q.
- **Hành động** (2): 0 = EXTEND (giữ pha xanh thêm) và 1 = SWITCH (chuyển sang pha vàng). Agent chỉ được hỏi mỗi 5 giây sau khi đã đạt thời gian xanh tối thiểu 10 giây; tại 60 giây buộc SWITCH để bảo đảm an toàn.
- **Thưởng** $r = -L$, với L là tổng chiều dài hàng đợi hiện tại — reward âm khuyến khích agent giải tỏa xe nhanh nhất có thể.

Sau mỗi lần quyết định, cặp (trạng thái trước, hành động trước) được cập nhật theo phương trình Bellman dạng bảng với tốc độ học $\alpha = 0{,}1$ và hệ số chiết khấu $\gamma = 0{,}9$:

$$Q(s,a) \leftarrow (1-\alpha)\, Q(s,a) + \alpha\Big(r + \gamma \max_{a'} Q(s',a')\Big). \tag{2}$$

Khám phá theo chiến lược ε-greedy: khởi tạo $\varepsilon_0 = 0{,}5$, giảm $\varepsilon \leftarrow 0{,}995\,\varepsilon$ *sau mỗi quyết định*, sàn $\varepsilon_{\min} = 0{,}05$; khi khai thác, chọn hành động có Q lớn hơn (nếu bằng nhau chọn ngẫu nhiên). Bảng Q được khởi tạo đầy đủ 18 trạng thái $\times$ 2 hành động bằng 0 ngay trong `init` của mô hình.

```
// Reflex qlearning_controller - tai moi diem quyet dinh (moi 5s)
curr_state = global_phase + "_" + level(queue_NS) + "_" + level(queue_EW);
// 1. Cap nhat Q cua buoc truoc (Bellman, reward = -tong hang doi)
old = Q[prev_state][prev_action];
Q[prev_state][prev_action] = (1-alpha)*old + alpha*(reward + gamma*max(Q[
    curr_state]));
// 2. Chon hanh dong theo epsilon-greedy
if (random() < epsilon) action = rnd(1);        // kham pha
else action = argmax(Q[curr_state]);            // khai thac
epsilon = max(eps_min, epsilon * 0.995);        // giam dan theo quyet dinh
// 3. Thuc thi: EXTEND -> giu xanh; SWITCH -> chuyen sang pha vang
```

Trong quá trình phát triển cũng phát hiện cùng lỗi headless như MaxPressure (danh sách đường rỗng khi `init` chạy $\Rightarrow$ hàng đợi luôn bằng 0 $\Rightarrow$ không gian trạng thái suy biến), và được khắc phục bằng cùng cơ chế khởi tạo muộn.

1.5 Deep Q-Learning — học tăng cường sâu

Tệp cài đặt: `intersection-dqn-python.gaml` (phía mô phỏng) và thư mục `dqn_py/` (phía Python/PyTorch): `gama_env.py` (môi trường giao thông dạng gym qua WebSocket), `dqn_agent.py` (agent DQN), `train.py` (vòng lặp huấn luyện), `evaluate.py` (đánh giá chính sách đã học).

Kiến trúc hai tiến trình

Khác với Q-Learning bảng, DQN cần bộ xấp xỉ hàm bằng mạng nơ-ron; do đó hệ thống được tách thành hai tiến trình giao tiếp qua **gama-server** (WebSocket, giao thức JSON, thư viện chính thức `gama-client`, cổng 6868):

- **GAMA** giữ toàn bộ logic thời gian và an toàn *giống hệt* mô hình Q-Learning bảng (xanh tối thiểu 10 s, tối đa 60 s, vàng 3 s, quyết định mỗi 5 s) để bảo đảm so sánh công bằng; nó cũng quan sát trạng thái (`queue_ns`, `queue_ew`, `approach_ns`, `approach_ew` — phân biệt xe đang dừng và xe đang chạy tới), xuất gói trạng thái và thực thi hành động của Python.
- **Python** chạy mạng DQN, nhận trạng thái, chọn hành động (0 = Extend, 1 = Switch) và huấn luyện theo kinh nghiệm.

Tại mỗi điểm quyết định, hai bên bắt tay (handshake) theo năm bước:

```
# Python (evaluate.py / train.py)  <->  GAMA (reflex dqn_controller)
state = await env.read_state()             # 1. GET state_pack (18 truong,
    noi boi "|")
await env.set_action(action)               # 2. SET dqn_action (0/1)
await env.step_cycles(1)                   # 3. GAMA ap dung action trong 1
    chu ky
nb = await env.get_next_decision_in()      # 4. Doc so chu ky den tick quyet
     dinh ke tiep
await env.step_cycles(nb)                  # 5. Chay dung den tick do (sync=
    True)
```

Phía GAMA, reflex `update_schedule` tính trước `next_decision_in_cycles` (phần còn lại của pha vàng + thời gian xanh tối thiểu, hoặc khoảng tick 5 giây kế tiếp trong pha xanh) giúp Python luôn bước mô phỏng đúng đến điểm quyết định mà không bỏ sót tick.

Không gian trạng thái và mạng nơ-ron

Trạng thái được nén thành **vector 8 chiều**, chuẩn hóa về $[0, 1]$ bởi hàm `featurize()` trong `gama_env.py`:

$$s = \left[\tfrac{q_{\mathrm{NS}}}{20},\ \tfrac{q_{\mathrm{EW}}}{20},\ \tfrac{a_{\mathrm{NS}}}{20},\ \tfrac{a_{\mathrm{EW}}}{20},\ \tfrac{t_{\mathrm{green}}}{600},\ \mathbb{1}[t_{\mathrm{green}} \geq 100],\ \mathbb{1}[\text{pha NS}],\ \mathbb{1}[\text{pha EW}]\right]$$

trong đó q là số xe đang dừng, a là số xe đang tiến gần trên đường vào, và t_{green} là số chu kỳ đã trôi trong pha xanh hiện tại. Mạng Q là MLP $8 \to 32 \to 32 \to 2$ (ReLU), huấn luyện bằng bộ thành phần chuẩn của DQN [3]: *experience replay* (bộ đệm 20 000 chuyển tiếp, cấp phát trước bằng numpy), *target network* đồng bộ mỗi 200 lần cập nhật, hàm mất Huber trên minibatch 64 mẫu, kẹp gradient (max-norm 10) và tối ưu Adam với tốc độ học 10^{-3}. Target Bellman: $y = r + \gamma \max_{a'} Q_{\mathrm{target}}(s', a')$ (với $y = r$ nếu kết thúc episode).

Reward và các siêu tham số học được đặt *trùng khớp* với mô hình Q-Learning bảng để so sánh công bằng: $r = -(w_q \cdot L_{\mathrm{queue}} + w_w \cdot \bar{t}_{\mathrm{wait}})$ với $(w_q, w_w) = (1{,}0;\ 0{,}0)$, $\gamma = 0{,}9$, $\varepsilon_0 = 0{,}5$, $\varepsilon \leftarrow 0{,}995\,\varepsilon$ mỗi quyết định, $\varepsilon_{\min} = 0{,}05$.

Bảng 1: So sánh cài đặt Q-Learning bảng và Deep Q-Learning

Thành phần	**Q-Learning bảng**	**Deep Q-Learning**
Biểu diễn trạng thái	18 trạng thái rời rạc	Vector 8 chiều $\in [0,1]^8$
Bộ xấp xỉ Q	Bảng `map<string, list>`	MLP 8→32→32→2 (PyTorch)
Hành động	EXTEND / SWITCH	Giống hệt bảng
Thưởng	$-L_{\text{queue}}$	$-(w_q L + w_w \bar{t})$, (w_q, w_w)=(1,0; 0,0)
γ / lr	0,9 / 0,1 (Bellman bảng)	0,9 / 10^{-3} (Adam)
Khám phá	ε-greedy 0,5→ × 0,995→0,05	Giống hệt bảng
Kỹ thuật huấn luyện	Cập nhật ngay mỗi quyết định	Replay 20 000, batch 64, target sync 200
Nơi thực thi	Trong tiến trình GAMA	Python (MPS/CUDA/CPU) qua gama-server

Huấn luyện, đánh giá và các sửa lỗi kỹ thuật

Huấn luyện (`train.py`): 150 episode, mỗi episode 2000 chu kỳ (200 giây), seed của episode e là $1000 + e$. Mỗi quyết định: featurize trạng thái → chọn hành động ε-greedy → handshake với GAMA → lưu chuyển tiếp $(s, a, r, s', done)$ vào replay → một bước gradient trên minibatch. Trọng số tốt nhất theo reward episode cao nhất được lưu vào `best_weights.pt`, kèm checkpoint định kỳ và nhật ký `training_log.csv`.

Đánh giá (`evaluate.py`): chạy lại mô phỏng với chính sách tham lam thuần ($\varepsilon = 0$) sử dụng `best_weights.pt` đã huấn luyện, ghi kết quả ra `dqn_metrics.csv` cùng schema với dòng `METRIC_CSV` của ba mô hình kia để ghép thẳng vào pipeline so sánh.

Hai lỗi kỹ thuật đáng chú ý đã được xử lý trong quá trình triển khai:

1. **Seed bị gama-server bỏ qua**: tham số đặc biệt `seed` truyền qua lệnh `reload` của gama-server không được áp dụng, khiến mọi episode chạy trùng nhau. Khắc phục bằng cách truyền seed như một tham số thường `rng_seed`; mô hình GAMA tự gán `seed` trong `init` trước khi sinh xe.
2. **Tick quyết định không tất định**: bộ đếm `phase_timer` dạng float (bước 0,1 s) tích lũy sai số làm tròn, khiến các tick 10 s, 15 s,… lệch bất định. Mô hình DQN thay bằng bộ đếm nguyên `green_cycles/yellow_cycles` (ngưỡng 100/600/30/50 chu kỳ) — tick quyết định hoàn toàn tất định trên lưới nguyên.

2 Kết quả benchmark và kết luận

2.1 Thiết lập thí nghiệm

Benchmark được điều phối bởi script `scripts/compare_4_models.py` với tham số `--runs 5`, tức **mỗi thuật toán chạy 5 lần độc lập**, mỗi lần 2000 chu kỳ (200 giây mô phỏng):

- **Fixed-Time, MaxPressure, Q-Learning**: chạy batch headless bằng lệnh `gama-headless.sh -batch my_batch_experiment <model.gaml>`; GAMA tự chọn seed cho từng lần chạy (tạo nhiễu tự nhiên giữa các lần), stdout được đọc trực tiếp và bóc tách các dòng `METRIC_CSV` (199 mẫu mỗi lần chạy, chu kỳ 10–1990).
- **Deep Q-Learning**: gama-server chạy sẵn ở cổng 6868; `evaluate.py` thực thi 5 episode với chính sách tham lam ($\varepsilon = 0$) dùng trọng số tốt nhất đã huấn luyện (`best_weights.pt` của lần chạy `train_20260829_153936`), seeds 5001–5005.

Với mỗi lần chạy, giá trị của sáu tiêu chí được quy đổi về một số: throughput và thời gian hành trình lấy *giá trị cuối*, thời gian chờ tối đa lấy *giá trị lớn nhất*, còn thời gian chờ trung bình /

hàng đợi / tốc độ lấy *trung bình theo thời gian*. Bảng kết quả tổng hợp dạng mean ± std trên 5 lần chạy; biểu đồ vẽ series đại diện (lần chạy có throughput cuối ở vị trí median) của từng thuật toán.

2.2 Kết quả tổng hợp

Bảng 2: Kết quả so sánh bốn thuật toán — mean ± std trên 5 lần chạy độc lập (mỗi lần 2000 chu kỳ = 200 giây). Throughput và tốc độ: cao hơn tốt hơn; các tiêu chí còn lại: thấp hơn tốt hơn. DQL: Deep Q-Learning.

Tiêu chí	**Fixed-Time**	**MaxPressure**	**Q-Learning**	**Deep Q-Learn.**	**Tốt nhất**
1. Throughput (xe/200 s)	111,20 ± 1,94	109,40 ± 2,87	**111,80 ± 4,26**	110,40 ± 3,14	Q-Learning
2. Thời gian chờ TB (s)	5,24 ± 0,44	3,63 ± 0,31	3,81 ± 0,37	**2,05 ± 0,10**	DQL
3. Hàng đợi TB (xe)	6,18 ± 0,52	4,61 ± 0,40	4,95 ± 0,43	**2,69 ± 0,12**	DQL
4. Tốc độ TB (km/h)	17,29 ± 0,44	18,56 ± 0,29	18,20 ± 0,16	**19,74 ± 0,23**	DQL
5. Thời gian hành trình (s)	47,69 ± 1,26	45,16 ± 0,53	46,13 ± 0,75	**43,43 ± 0,56**	DQL
6. Chờ tối đa (s)	36,54 ± 1,52	31,50 ± 4,67	28,78 ± 2,71	**16,26 ± 0,94**	DQL
Số tiêu chí thắng (theo mean): Fixed-Time: 0		*MaxPressure: 0*	*Q-Learning: 1*	*Deep Q-Learning: 5*	



Hình 1: Diễn biến sáu tiêu chí theo thời gian mô phỏng (series đại diện, 200 giây): Fixed-Time (đỏ), MaxPressure (xanh lá), Q-Learning (xanh dương), Deep Q-Learning (tím, nét đứt).

2.3 Phân tích kết quả

Throughput (tiêu chí 1). Cả bốn thuật toán cho throughput gần nhau (107–119 xe/200 giây) và các đường throughput trong Hình 1 gần như trùng nhau. Điều này dễ hiểu: luồng sinh xe là nút thắt của hệ thống, nên mọi bộ điều khiển đều kịp giải tỏa toàn bộ demand. Khác biệt thật sự nằm ở *chi phí* phải trả để đạt được cùng lượng xe qua ngã tư.

Chờ trung bình và hàng đợi (tiêu chí 2–3). Fixed-Time tệ nhất với 5,24 giây chờ và 6,18 xe hàng đợi trung bình, với các đợt dồn ứ rõ rệt trên biểu đồ (hàng đợi có lúc đạt đỉnh ∼20 xe, chờ trung bình đỉnh ∼10 giây) trùng với lúc trục đông bị trì hoãn bởi chu kỳ cố định. Hai thuật toán thích ứng giảm mạnh: MaxPressure 3,63 giây (−31%), Q-Learning 3,81 giây (−27%). Deep Q-Learning tốt nhất với 2,05 giây (−61% so với Fixed-Time) và hàng đợi 2,69 xe (−56%) — nhờ mạng nơ-ron quan sát được cả lưu lượng *đang tiến gần* (approach), cho phép chủ động chuyển pha trước khi hàng đợi kịp hình thành.

Tốc độ và thời gian hành trình (tiêu chí 4–5). Thứ tự tương tự: DQN đạt tốc độ trung bình cao nhất 19,74 km/h (+14% so với Fixed-Time) và thời gian hành trình ngắn nhất 43,43 giây (−9%). Ba đường travel time hội tụ về gần nhau ở cuối mô phỏng, cho thấy advantage của RL chủ yếu nằm ở pha khởi động khi demand đang tăng.

Công bằng (tiêu chí 6). Đây là tiêu chí chênh lệch rõ nhất: thời gian chờ tối đa của Fixed-Time là 36,54 giây, của DQN chỉ còn 16,26 giây (−55%). Đáng chú ý, DQN còn có độ lệch chuẩn nhỏ nhất (0,94) trên 5 lần chạy — chính sách học được không chỉ tốt hơn mà còn *ổn định*; ngược lại MaxPressure có std lớn (4,67) do chính sách nhạy theo tức thời của áp lực.

Đánh giá chung. Deep Q-Learning thắng 5/6 tiêu chí; Q-Learning bảng chỉ thắng throughput (với std lớn nhất 4,26 — kết quả dao động nhiều giữa các lần chạy). MaxPressure tốt hơn Fixed-Time ở mọi tiêu chí còn lại, khẳng định giá trị của điều khiển thích ứng. Thứ tự tổng thể: **Deep Q-Learning** > **MaxPressure** ≈ **Q-Learning** > **Fixed-Time**.

2.4 Kết luận và hướng phát triển

Bốn bộ điều khiển đèn giao thông đã được triển khai hoàn chỉnh và đánh giá trên cùng một nền tảng mô phỏng GAMA, cùng cấu hình ngã tư và cùng luồng sinh xe. Kết luận chính:

- **Deep Q-Learning là phương pháp tốt nhất** trong thí nghiệm này: giảm 61% thời gian chờ trung bình, 56% hàng đợi, 55% thời gian chờ tối đa và tăng 14% tốc độ so với baseline Fixed-Time, đồng thời có phương sai nhỏ nhất giữa các lần chạy. Đánh đổi là hệ thống phức tạp hơn (huấn luyện trước với PyTorch, hạ tầng gama-server/WebSocket) và cần thời gian huấn luyện 150 episode.
- **MaxPressure là lựa chọn thực dụng tốt**: không cần huấn luyện, cài đặt trong 100 dòng GAML, hiệu năng chỉ thấp hơn DQN và cao hơn hẳn Fixed-Time — phù hợp khi muốn triển khai nhanh, không cần hạ tầng học máy.
- **Q-Learning bảng học được chính sách tốt** trong không gian 18 trạng thái nhỏ, nhưng bị giới hạn bởi việc rời rạc hóa trạng thái (mất thông tin về lưu lượng tiến gần) và nhiễu cao giữa các lần chạy.
- **Fixed-Time chịu ảnh hưởng nặng của demand bất đối xứng**: với chu kỳ 66 giây cứng, trục đông bị dồn ứ trong khi trục vắng lãng phí thời gian xanh, thể hiện qua cả ba tiêu chí chờ/hàng đợi/công bằng.

Các hướng phát triển tiếp theo: (i) kiểm định thống kê chính thức (ví dụ kiểm định Wilcoxon rank-sum) thay vì so sánh mean–std; (ii) mở rộng sang mạng nhiều ngã tư với multi-agent RL; (iii) phát triển hàm thưởng (thêm trọng số thời gian chờ, độ công bằng); (iv) thử nghiệm demand biến động theo thời gian và demand bão hòa; (v) so sánh thêm với baseline Webster và các biến thể DQN (Double DQN, Dueling).

Tài liệu

[1] P. Varaiya, "Max pressure control of a network of signalized intersections," *Transportation Research Part C: Emerging Technologies*, vol. 36, pp. 177–195, 2013.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[3] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[4] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Physical Review E*, vol. 62, no. 2, pp. 1805–1824, 2000.

[5] P. Taillandier *et al.*, "Building, composing and experimenting complex spatial models with the GAMA platform," *GeoInformatica*, vol. 23, no. 2, pp. 299–322, 2019.